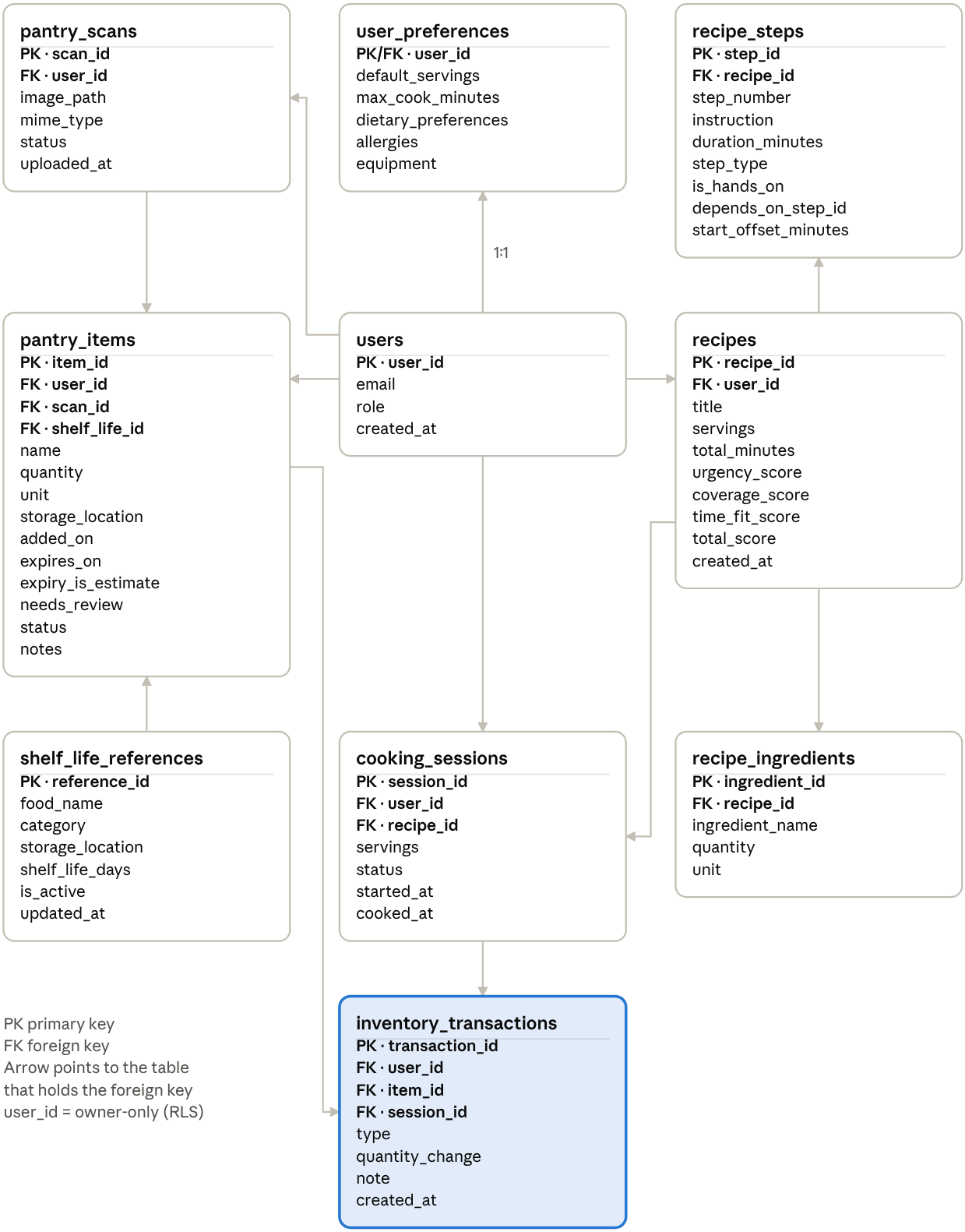
FridgeFirst: Database Design

Oct 7, 2026 · @Samia

Entity relationship diagram

Pantry quantities change only through recorded inventory transactions



entity relationship diagram · 10 tables, 12 relationships

Each of the design doc's seven entities is a table in the diagram, and three supporting tables hold what its requirements ask for: user_preferences (FR 7), recipe_ingredients and recipe_steps (FR 9 and 10). Pantry quantities never change directly: every addition, correction, deduction and discard writes a row to inventory_transactions, which is the highlighted table. The database is Supabase PostgreSQL, and every table that carries a user_id is restricted to its owner by Row Level Security.

What we store

Ten tables hold everything the design doc requires. Types are PostgreSQL types. FR and NFR numbers refer to the design doc's requirement tables.

users (profile row for each Supabase Auth account)

Field	Type	Notes
user_id	uuid	Primary key; the same id Supabase Auth issues (FR 1). Passwords stay in Supabase Auth, not here
email	text	Unique
role	text	user or admin; only admins can edit shelf_life_references (FR 13)
created_at	timestamptz	

user_preferences (one row per user)

Field	Type	Notes
user_id	uuid	Primary key and foreign key to users
default_servings	integer	FR 7
max_cook_minutes	integer	FR 7; feeds the time-fit score
dietary_preferences	text[]	FR 7
allergies	text[]	FR 7; applied as a hard filter on recipes
equipment	text[]	FR 7

pantry_scans (one row per uploaded image)

Field	Type	Notes
scan_id	uuid	Primary key
user_id	uuid	Foreign key to users
image_path	text	Object path in the private Supabase Storage bucket
mime_type	text	JPEG, PNG or HEIC (FR 2)
status	text	uploaded, analyzed, confirmed or failed
uploaded_at	timestamptz	

shelf_life_references (shared reference data, not owned by any user)

Field	Type	Notes
reference_id	uuid	Primary key
food_name	text	
category	text	
storage_location	text	Shelf life differs by where food is kept
shelf_life_days	integer	Used to estimate expires_on (FR 6)
is_active	boolean	Admins deactivate and restore entries instead of deleting them (FR 13)
updated_at	timestamptz	

pantry_items

Field	Type	Notes
item_id	uuid	Primary key
user_id	uuid	Foreign key to users

Field	Type	Notes
scan_id	uuid	Foreign key to pantry_scans; empty when the item was added by hand
shelf_life_id	uuid	Foreign key to shelf_life_references; empty when no reference matched
name	text	FR 5
quantity	numeric	Current amount; changes only alongside an inventory_transactions row
unit	text	FR 5
storage_location	text	FR 5
added_on	date	FR 5
expires_on	date	FR 5; the date the urgency score is based on
expiry_is_estimate	boolean	True until the user sets the date; labels it as an estimate (FR 6, NFR 6)
needs_review	boolean	Surfaces the items that require review on the dashboard (FR 12)
status	text	active, consumed or discarded (FR 4)
notes	text	FR 5

recipes (recommendations generated for one user)

Field	Type	Notes
recipe_id	uuid	Primary key
user_id	uuid	Foreign key to users
title	text	
servings	integer	
total_minutes	integer	Total time (FR 9)
urgency_score	numeric	Share of the ranking based on how soon ingredients expire

Field	Type	Notes
coverage_score	numeric	Share based on how many ingredients the pantry already has
time_fit_score	numeric	Share based on fit with the user's time limit
total_score	numeric	Weighted sum, initially 50 percent, 30 percent and 20 percent; the weights live in configuration so they can change after usability testing
created_at	timestamptz	

recipe_ingredients

Field	Type	Notes
ingredient_id	uuid	Primary key
recipe_id	uuid	Foreign key to recipes
ingredient_name	text	Compared with pantry_items to list what is missing (FR 8)
quantity	numeric	FR 9
unit	text	FR 9

recipe_steps (numbered instructions and the parallel timeline)

Field	Type	Notes
step_id	uuid	Primary key
recipe_id	uuid	Foreign key to recipes
step_number	integer	Numbered instructions (FR 9)
instruction	text	
duration_minutes	integer	
step_type	text	cook or cleanup; cleanup tasks are placed only in waiting periods (FR 10)

Field	Type	Notes
is_hands_on	boolean	False during waiting periods such as simmering or baking
depends_on_step_id	uuid	Foreign key to recipe_steps; a step that must finish first. Empty for steps that can overlap
start_offset_minutes	integer	When the step starts in the parallel timeline

cooking_sessions

Field	Type	Notes
session_id	uuid	Primary key
user_id	uuid	Foreign key to users
recipe_id	uuid	Foreign key to recipes
servings	integer	
status	text	in progress, cooked or cancelled
started_at	timestamptz	
cooked_at	timestamptz	Set when the user confirms the deductions (FR 11)

inventory_transactions (the audit trail)

Field	Type	Notes
transaction_id	uuid	Primary key
user_id	uuid	Foreign key to users
item_id	uuid	Foreign key to pantry_items
session_id	uuid	Foreign key to cooking_sessions; empty unless the change came from cooking
type	text	addition, correction, recipe_deduction, consumed or discarded
quantity_change	numeric	Negative when food is used up

Field	Type	Notes
note	text	
created_at	timestamptz	The recent inventory changes shown on the dashboard (FR 12)

How the tables connect

The diagram draws twelve relationships, and a thirteenth is described below because drawing it would cross the rest. All are one-to-many except users to user_preferences, which is one-to-one.

Relationship	Type	Why the app needs it
users to user_preferences	One to one	Each person keeps their own servings, time limit, diet, allergies and equipment (FR 7)
users to pantry_scans	One to many	A user uploads many photos over time (FR 2)
users to pantry_items	One to many	Each user has a private pantry (FR 4, NFR 3)
users to recipes	One to many	Recommendations are generated from one user's pantry (FR 8)
users to cooking_sessions	One to many	A user cooks many times (FR 11)
pantry_scans to pantry_items	One to many, optional	Items detected in a photo link back to it; items added by hand have no scan
shelf_life_references to pantry_items	One to many, optional	Supplies the shelf life behind an estimated expiration date (FR 6)
pantry_items to inventory_transactions	One to many	Every quantity change is explained by a transaction row
cooking_sessions to inventory_transactions	One to many, optional	Groups the deductions made by one cooked recipe
recipes to recipe_ingredients	One to many	Ingredients and quantities (FR 9)

Relationship	Type	Why the app needs it
recipes to recipe_steps	One to many	Numbered instructions and the parallel timeline (FR 9, FR 10)
recipes to cooking_sessions	One to many	Each time a recipe is cooked is recorded separately
users to inventory_transactions	One to many	Not drawn. The user_id on each transaction lets Row Level Security restrict rows to their owner

Marking a recipe as cooked. The app shows the proposed deductions, and after the user confirms it writes everything in one database transaction: it sets the cooking session to cooked, inserts one inventory_transactions row of type recipe_deduction for each affected item, and lowers each pantry_items.quantity by the same amount. If any step fails, none of it is saved (NFR 5).

Ranking recipes. Urgency comes from the expires_on dates of the pantry items that match a recipe's ingredients, coverage from how many ingredients the pantry has, and time fit from max_cook_minutes. The three results are saved on the recipe, so the same ranking can be shown again without recomputing it.

Recipe steps. depends_on_step_id points to another row in recipe_steps, so a step can wait for an earlier step while independent steps overlap. Cleanup steps are flagged with step_type and placed only where is_hands_on is false.

How this follows the design doc

Each of the design doc's seven primary entities is one table, and the three extra tables exist only because a functional requirement needs somewhere to store its data.

Design doc	Table	Requirements covered
User	users, user_preferences	FR 1, FR 7, FR 13 (role)
PantryScan	pantry_scans	FR 2, FR 3
PantryItem	pantry_items	FR 4, FR 5, FR 6, FR 12
ShelfLifeReference	shelf_life_references	FR 6, FR 13
Recipe	recipes, recipe_ingredients, recipe_steps	FR 8, FR 9, FR 10

Design doc	Table	Requirements covered
CookingSession	cooking_sessions	FR 11
InventoryTransaction	inventory_transactions	FR 4, FR 11, FR 12, NFR 5

The design doc does not specify these, so I chose them. Change any that your team decides differently:

- **One scan row per image.** FR 2 allows several images in one upload, so a multi-photo upload creates several pantry_scans rows.
- **Recipes belong to one user.** The design doc has the AI draft recipes from a user's confirmed pantry, so each recipe is saved for that user rather than shared.
- **Missing ingredients are calculated, not stored.** FR 8 needs them listed, and comparing recipe_ingredients with the pantry gives the list.
- **Ranking weights are not in the database.** The design doc calls them configurable, so they sit in application configuration.
- **Ids are uuid.** This matches the Supabase Auth user id, which users.user_id reuses.

Changes from my first draft. That draft followed the wireframes and the pitch, which describe a household-sharing app. This version removes households and shared pantries, barcode and receipt entry, and the alerts table, because the design doc lists shared household pantries and barcode scanning as future work. It adds photo scans, cooking sessions and the timeline steps. The earlier usage_log became inventory_transactions.